Here is the implementation for **Stage 3: profile route token verification**. We now pass the extracted token to Supabase to verify its authenticity.

Node.js (Express & JavaScript)

javascript

```javascript
// Update your GET /protected/profile route from Stage 2

app.get('/protected/profile', async (req, res) => {
  const authHeader = req.headers.authorization;

  // 1. Extract and validate header presence
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: "Access token required" });
  }

  const parts = authHeader.split(' ');
  const token = parts[1];

  if (!token) {
    return res.status(401).json({ error: "Access token required" });
  }

  // 2. Ask Supabase if the token is authentic
  const { data: { user }, error } = await supabase.auth.getUser(token);

  // 3. Turn away expired, tampered, or invalid tokens
  if (error || !user) {
    return res.status(401).json({ error: "Invalid or expired token" });
  }

  // 4. Return safe user metadata on success
  return res.status(200).json({
    id: user.id,
    email: user.email,
    created_at: user.created_at
  });
});
```

Use code with caution.

Python (FastAPI)

python

```python
from fastapi import Request, HTTPException, status
from fastapi.responses import JSONResponse

# Update your GET /protected/profile route from Stage 2

@app.get("/protected/profile")
def get_protected_profile(request: Request):
    auth_header = request.headers.get("Authorization")

    # 1. Extract and validate header presence
    if (!auth_header or not auth_header.startswith("Bearer ")):
        return JSONResponse(
            status_code=status.HTTP_401_UNAUTHORIZED,
            content={"error": "Access token required"}
        )

    try:
        parts = auth_header.split(" ")
        token = parts[1]
    except IndexError:
        return JSONResponse(
            status_code=status.HTTP_401_UNAUTHORIZED,
            content={"error": "Access token required"}
        )

    # 2. Ask Supabase if the token is authentic
    try:
        response = supabase.auth.get_user(token)
        user = response.user

        if not user:
            raise ValueError()

    # 3. Turn away expired, tampered, or invalid tokens
    except Exception:
        return JSONResponse(
            status_code=status.HTTP_401_UNAUTHORIZED,
            content={"error": "Invalid or expired token"}
        )
```

```
    # 4. Return safe user metadata on success
    return {
        "id": user.id,
        "email": user.email,
        "created_at": user.created_at
    }
```

Use code with caution.

Now that you have verified credentials coming through, let me know what you would like to tackle next:

- Refactor the verification logic into an automated **middleware function**
- Create a **database table** in Supabase and hook up row-level security (RLS)
- Implement a **token refresh** endpoint using the refresh token